UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

GUÍA DE LABORATORIO N.º 07

Primalidad, grupos y algoritmos probabilísticos

Asignatura:	Algoritmos Avanzados
Unidad:	II. Fundamentos algebraicos y algoritmos probabilísticos
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Equipos del proyecto semestral, aproximadamente cuatro estudiantes
Lenguaje:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica **2**

2. Trabajo preparatorio obligatorio **2**

3. Materiales **2**

4. Marco teórico breve **2**

4.1. División de prueba . . . 2

4.2. Fermat y pseudoprimos . . . 2

4.3. Miller–Rabin . . . 3

5. Especificación mínima **3**

6. Propiedades que deben comprobarse **3**

7. Distribución sugerida de los 120 minutos **3**

8. Ejercicios resueltos **3**

8.1. Ejercicio resuelto 1: división de prueba y Fermat . . . 3

8.2. Ejercicio resuelto 2: Miller–Rabin con traza . . . 4

8.3. Ejercicio resuelto 3: prueba repetida y candidatos . . . 5

9. Ejercicios propuestos **5**

9.1. Ejercicio propuesto 1: biblioteca de primalidad . . . 5

9.2. Ejercicio propuesto 2: comparación experimental . . . 5

10. Metodología de medición **6**

11. Resultados mínimos requeridos **6**

12. Preguntas de análisis **6**

13. Producto de entrega **7**

14. Tabla de evaluación **7**

15. Casos mínimos de prueba **7**

16. Referencias **8**

1. Objetivo de la práctica

Objetivo

Implementar y comparar pruebas de primalidad deterministas y probabilísticas, analizar pseudoprimos y números de Carmichael, ejecutar Miller–Rabin con trazas verificables y generar probables primos de una longitud binaria especificada.

Al finalizar, el equipo podrá:

1. implementar división de prueba y Fermat;
2. explicar por qué Fermat puede aceptar compuestos;
3. descomponer $n - 1 = 2^s d$ y ejecutar Miller–Rabin;
4. distinguir testigos, mentirosos fuertes y probables primos;
5. estimar la cota de error 4^{-k};
6. generar candidatos impares de L bits;
7. validar experimentalmente corrección, rendimiento y reproducibilidad.

2. Trabajo preparatorio obligatorio

Cada integrante deberá revisar $\mathbb{Z}_n^*$, función φ, orden de un elemento, teoremas de Euler y Fermat, exponenciación modular, representación binaria y diferencias entre algoritmos Monte Carlo y Las Vegas.

3. Materiales

- Python 3.11 o posterior, editor, terminal y Git.
- Biblioteca estándar: `math`, `random`, `secrets`, `time`, `statistics` y `csv`.
- Diapositivas del tema 7 y repositorio del equipo.
- Herramienta para elaborar tablas y gráficos desde CSV.

4. Marco teórico breve

4.1. División de prueba

Si n es compuesto, tiene un factor no mayor que $\sqrt{n}$. Probar divisores hasta $\sqrt{n}$ cuesta $\mathcal{O}(\sqrt{n})$ operaciones aritméticas en el modelo elemental, lo que es exponencial respecto de la longitud binaria.

4.2. Fermat y pseudoprimos

Si p es primo y $p \nmid a$, entonces $a^{p-1} \equiv 1 \pmod{p}$. El recíproco es falso: 341 es compuesto, pero $2^{340} \equiv 1 \pmod{341}$. Los números de Carmichael, como 561, superan Fermat para toda base coprima.

4.3. Miller–Rabin

Para un impar $n > 2$, se escribe $n-1 = 2^s d$, con d impar. Para una base a, se calcula $x = a^d$ mód n y se examina la cadena de cuadrados. Un testigo demuestra composición; superar k rondas independientes solo permite declarar probable primo, con cota de error 4^{-k} para un compuesto impar.

5. Especificación mínima

Listing 1: Interfaz requerida

```python
def trial_division(n): ...
def fermat_round(n, a): ...
def decompose(n): ...
def miller_rabin_round(n, a, trace=False): ...
def is_probable_prime(n, rounds=20, rng=None): ...
def generate_candidate(bits, rng): ...
def generate_probable_prime(bits, rounds=20): ...
```

6. Propiedades que deben comprobarse

1. la división de prueba coincide con un verificador conocido en rangos pequeños;
2. si Fermat falla, el número es compuesto;
3. `decompose` satisface $n - 1 = 2^s d$ y d impar;
4. si Miller–Rabin falla, la base es un testigo concluyente;
5. todo primo pequeño supera todas las bases válidas;
6. el candidato generado tiene exactamente L bits y es impar;
7. la ejecución experimental es reproducible con la misma semilla;
8. el resultado “probable primo” informa el número de rondas y la cota de error.

7. Distribución sugerida de los 120 minutos

Etapa	Actividad	Tiempo
Inicio	Interfaces, casos y convenciones	10 min
Resuelto 1	División de prueba y Fermat	20 min
Resuelto 2	Miller–Rabin con traza	25 min
Resuelto 3	Generación de candidatos	15 min
Propuesto 1	Biblioteca integrada y validación	28 min
Propuesto 2	Experimentos y análisis	17 min
Cierre	Revisión individual	5 min

8. Ejercicios resueltos

8.1. Ejercicio resuelto 1: división de prueba y Fermat

```python
from math import isqrt, gcd

def trial_division(n):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    limit = isqrt(n)
    for d in range(3, limit + 1, 2):
        if n % d == 0:
            return False
    return True

def fermat_round(n, a):
    if not 2 <= a <= n - 2:
        raise ValueError("Base fuera de rango")
    if gcd(a, n) > 1:
        return False
    return pow(a, n - 1, n) == 1

assert trial_division(221) is False
assert fermat_round(15, 2) is False
assert fermat_round(341, 2) is True
```

El último caso muestra que superar Fermat no prueba primalidad: $341 = 11 \cdot 31$ es pseudoprimo base 2.

8.2. Ejercicio resuelto 2: Miller–Rabin con traza

```python
def decompose(n):
    s, d = 0, n - 1
    while d % 2 == 0:
        s += 1
        d //= 2
    return s, d

def miller_rabin_round(n, a, trace=False):
    s, d = decompose(n)
    x = pow(a, d, n)
    values = [x]
    if x in (1, n - 1):
        return True, values
    for _ in range(s - 1):
        x = (x * x) % n
        values.append(x)
        if x == n - 1:
            return True, values
        if x == 1:
            return False, values
    return False, values

passed, values = miller_rabin_round(221, 2, True)
assert passed is False
assert values == [128, 30]
```

Como $220 = 2^2 \cdot 55$, la cadena empieza en $2^{55} \bmod 221 = 128$ y continúa con $128^2 \bmod 221 = 30$. No aparece 220, por lo que 2 es testigo de composición.

8.3. Ejercicio resuelto 3: prueba repetida y candidatos

```python
from random import Random
from secrets import randbelow, randbits

def is_probable_prime(n, rounds=20, rng=None):
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0: return False
    for _ in range(rounds):
        a = (rng.randrange(2, n - 1) if rng
             else 2 + randbelow(n - 3))
        passed, _ = miller_rabin_round(n, a)
        if not passed:
            return False
    return True

def generate_candidate(bits, rng=None):
    r = rng.getrandbits(bits) if rng else randbits(bits)
    return r | (1 << (bits - 1)) | 1

rng = Random(2026)
n = generate_candidate(32, rng)
assert n.bit_length() == 32 and n % 2 == 1
```

La semilla fija se utiliza para experimentos reproducibles. La fuente segura del sistema se reserva para generación con propósito criptográfico.

9. Ejercicios propuestos

9.1. Ejercicio propuesto 1: biblioteca de primalidad

Construya `primality.py` integrando los algoritmos anteriores. Debe incluir precribado por primos pequeños, Fermat con bases configurables, Miller–Rabin con traza opcional, generación de candidatos y generación de probables primos. La salida de una ronda debe incluir n, a, s, d y la secuencia de residuos. Valide obligatoriamente: $0, 1, 2, 3$, pares, primos pequeños, cuadrados de primos, 341, 561, 1105, 1729 y al menos 1000 enteros aleatorios menores que 10^6 comparados con división de prueba. Para rangos pequeños, clasifique cada base válida como testigo o mentiroso fuerte.

9.2. Ejercicio propuesto 2: comparación experimental

Realice experimentos reproducibles con longitudes de 16, 32, 64, 128, 256, 512 y 1024 bits. Para cada tamaño registre candidatos, descartes por precribado, rondas ejecutadas, cuadrados modulares, tiempo y resultado. Compare: división de prueba donde sea viable, Fermat y Miller–Rabin con 1, 5, 10 y 20 rondas. En un rango pequeño calcule la tasa empírica de compuestos aceptados y compárela con la cota teórica; no interprete la ausencia de errores observados como demostración de error cero. Genere además al menos 30 probables primos por tamaño y analice el número de candidatos requeridos.

Interpretación obligatoria

Separar corrección, probabilidad de error y rendimiento. Una salida “compuesto” de Miller–Rabin es concluyente; una salida “probable primo” depende de las bases y rondas ejecutadas.

10. Metodología de medición

```python
from statistics import median
from time import perf_counter_ns

def measure(function, repetitions=10):
    times = []
    for _ in range(repetitions):
        start = perf_counter_ns()
        function()
        times.append(perf_counter_ns() - start)
    return min(times), median(times), max(times)
```

Genere entradas antes de medir, use iguales entradas para cada método, reporte semilla, mediana, mínimo y máximo, y no incluya impresión o escritura de archivos en el intervalo medido.

11. Resultados mínimos requeridos

1. tabla de casos conocidos y clasificación;
2. trazas de 221, 341 y 561 con bases indicadas;
3. tabla de testigos y mentirosos fuertes en un rango pequeño;
4. gráfico de tiempo frente a longitud binaria;
5. gráfico de rondas o cuadrados modulares;
6. comparación de divisiones de prueba, Fermat y Miller–Rabin;
7. tabla de candidatos requeridos para generar probables primos;
8. CSV con semillas, parámetros y resultados.

12. Preguntas de análisis

1. ¿Por qué basta probar divisores hasta $\sqrt{n}$?
2. ¿Por qué esa cota es exponencial en los bits?
3. ¿Por qué Fermat puede demostrar composición, pero no primalidad?
4. ¿Qué diferencia hay entre pseudoprimo y número de Carmichael?
5. ¿Por qué Miller–Rabin examina $n - 1 = 2^s d$?
6. ¿Qué significa encontrar una raíz no trivial de 1?
7. ¿Qué diferencia hay entre testigo y mentiroso fuerte?
8. ¿Por qué el error es unilateral?
9. ¿Qué hipótesis permite usar 4^{-k}?
10. ¿Por qué una semilla fija no es apropiada para generar claves?
11. ¿Cómo influye el precribado?
12. ¿Por qué “no observamos errores” no significa “el algoritmo no puede errar”?

13. Producto de entrega

Producto de entrega

1. código organizado de los tres ejercicios resueltos y dos propuestos;
2. biblioteca `primality.py`;
3. pruebas automatizadas y generador reproducible;
4. CSV y gráficos;
5. informe PDF con ejercicios, metodología, resultados y respuestas;
6. conclusiones grupales de 250 a 350 palabras.

14. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación	Asiste, permanece y participa en implementación, pruebas y análisis; se determina individualmente.	4
Corrección y funcionamiento	División de prueba, Fermat, Miller–Rabin y generación funcionan; las trazas y pruebas verifican propiedades y casos especiales.	8
Análisis de complejidad y resultados	Relaciona bits, rondas, operaciones, cota de error y tiempos, interpretando correctamente la evidencia.	3
Dominio individual	Explica y ejecuta algoritmos, modifica una prueba y justifica una conclusión durante la revisión.	4
Total		**19**

15. Casos mínimos de prueba

- $n < 2$, 2, 3, pares e impares;
- primos y compuestos pequeños;
- cuadrados de primos;
- 341, 561, 1105 y 1729;
- bases en los extremos permitidos;
- distintas cantidades de rondas;
- candidatos de 16 a 1024 bits;
- repetición con la misma semilla;
- generación segura sin semilla fija.

16. Referencias

Referencias

[1] T. H. Cormen et al. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[2] V. Shoup. *A Computational Introduction to Number Theory and Algebra*, 2.ª ed. Cambridge University Press, 2009.

[3] R. Crandall y C. Pomerance. *Prime Numbers: A Computational Perspective*, 2.ª ed. Springer, 2005.

[4] Python Software Foundation. *secrets — Generate secure random numbers*. https://docs.python.org/3/library/secrets.html